

🔧 ИСПРАВЛЕНИЕ QML ↔ C++ СИНХРОНИЗАЦИИ

❌ ПРОБЛЕМА:

1. QML создаёт mock клипы вместо реальных
 2. Timeline.h НЕ ХВАТАЕТ функции getClipsForTrack()
 3. ClipManager.qml и Track.qml не синхронизированы с C++
-

✅ РЕШЕНИЕ:

1. ДОБАВЬ В timeline.h (после applyEffect):

```
cpp

// Применить эффект к клипу
Q_INVOKABLE bool applyEffect(int index, const QString& effectName, double value);

// ===== ПОЛУЧЕНИЕ КЛИПОВ ДЛЯ QML =====

// Получить все клипы для указанной дорожки (для отображения в QML)
Q_INVOKABLE QVariantList getClipsForTrack(int trackIndex);

// ===== ПРОВЕРКА ПЕРЕСЕЧЕНИЙ =====
```

Также добавь в начало файла:

```
cpp

#include <QVariantList>
#include <QFileInfo>
```

2. ДОБАВЬ В timeline.cpp (в начало, после includes):

```
cpp
```

```
#include <QFileInfo>
```

```
// После конструктора Timeline добавь:
```

```
// ===== ПОЛУЧИТЬ КЛИПЫ ДЛЯ ДОРОЖКИ (для QML) =====
```

```
QVariantList Timeline::getClipsForTrack(int trackIndex) {
```

```
    QVariantList result;
```

```
    for (int i = 0; i < m_clips.size(); ++i) {
```

```
        const TimelineClip& clip = m_clips[i];
```

```
        if (clip.trackIndex == trackIndex) {
```

```
            QVariantMap clipMap;
```

```
            // ID для QML
```

```
            clipMap["id"] = i;
```

```
            // Основные данные
```

```
            clipMap["filepath"] = clip.filepath;
```

```
            clipMap["startTime"] = clip.startTime;
```

```
            clipMap["duration"] = clip.duration;
```

```
            // Имя файла (без пути)
```

```
            QFileInfo fileInfo(clip.filepath);
```

```
            clipMap["filename"] = fileInfo.fileName();
```

```
            // Trim
```

```
            clipMap["trimStart"] = clip.trimStart;
```

```
            clipMap["trimEnd"] = clip.trimEnd;
```

```
            // Аудио
```

```
            clipMap["audioOffset"] = clip.audioOffset;
```

```
            clipMap["isMuted"] = clip.isMuted;
```

```
            // Эффекты
```

```
            QVariantMap effectsMap;
```

```
            for (auto it = clip.effects.begin(); it != clip.effects.end(); ++it) {
```

```
                effectsMap[it.key()] = it.value();
```

```
            }
```

```
            clipMap["effects"] = effectsMap;
```

```
            // Состояние
```

```
            clipMap["selected"] = false;
```

```
            clipMap["thumbnailPath"] = "";
```

```
            result.append(clipMap);
```

```
}  
}  
  
qDebug() << " 📄 getClipsForTrack" << trackIndex << "→" << result.size() << "клипов";  
return result;  
}
```

3. ЗАМЕНИ ClipManager.qml ПОЛНОСТЬЮ:

qml

```

import QtQuick
import "../theme.js" as Theme

QtObject {
    id: root

    // Сигналы
    signal clipAdded(string filepath, int trackNumber, double startTime)

    // ===== ДОБАВИТЬ КЛИП (ДЕЛЕГИРУЕМ В C++!) =====
    function addClip(filepath, trackNumber, startTime) {
        console.log("👉 ClipManager: Вызываю cppTimeline.addClip")
        console.log("  filepath:", filepath)
        console.log("  trackNumber:", trackNumber)
        console.log("  startTime:", startTime)

        // ❌ УБРАЛИ ВСЕ МОСК КОД!

        // ✅ Проверка расширения файла
        var ext = filepath.split('.').pop().toLowerCase()
        var videoExts = ['mp4', 'avi', 'mov', 'mkv', 'webm', 'flv', 'm4v', 'wmv']

        if (videoExts.indexOf(ext) === -1) {
            console.log("❌ Не видео файл:", ext)
            return
        }

        // ✅ ВЫЗЫВАЕМ C++!
        var success = cppTimeline.addClip(filepath, trackNumber, startTime)

        if (success) {
            console.log("✅ C++ успешно добавил клип!")
        } else {
            console.log("❌ C++ не смог добавить клип")
        }

        // C++ сам emit clipsChanged() → Track.qml обновится!
    }

    // ===== СЛУШАЕМ СИГНАЛЫ ОТ C++ =====
    Connections {
        target: cppTimeline

        function onClipsChanged() {
            console.log("👈 ClipManager получил: clipsChanged от C++")
            // Track.qml автоматически обновится!
        }
    }
}

```

```
}

function onClipAdded(index) {
  console.log(" 🎬 Клип добавлен, индекс:", index)
}

function onClipRemoved(index) {
  console.log(" 🎬 Клип удалён, индекс:", index)
}

function onTotalDurationChanged() {
  console.log(" 🎬 Длительность timeline изменилась")
}
}
}
```

4. ИСПРАВЬ Track.qml - найди строки с Repeater и замени:

НАЙДИ:

```
qml

// Repeater для клипов (МОСК)
Repeater {
  model: root.clips // МОСК clips!
```

ЗАМЕНИ НА:

```
qml
```

```
// ===== ПОЛУЧЕНИЕ CLIPS ИЗ C++ =====
property var realClips: [] // Реальные клипы из C++

// Обновить clips из C++
function updateClipsFromCpp() {
  console.log(" 🔄 Track", root.trackNumber, "updateClipsFromCpp")
  root.realClips = cppTimeline.getClipsForTrack(root.trackNumber)
  console.log(" Получено клипов:", root.realClips.length)
}

// Слушаем изменения
Connections {
  target: cppTimeline

  function onClipsChanged() {
    console.log(" 📡 Track", root.trackNumber, "получил clipsChanged")
    root.updateClipsFromCpp()
  }
}

// Инициализация
Component.onCompleted: {
  console.log(" ✅ Track", root.trackNumber, "создан")
  root.updateClipsFromCpp()
}

// ===== REPEATER ДЛЯ CLIPS (РЕАЛЬНЫЕ ИЗ C++!) =====
Repeater {
  model: root.realClips // ← РЕАЛЬНЫЕ ИЗ C++!
```

И внутри Repeater замени:

qml

```
delegate: Loader {
    source: "../components/VideoClip.qml"

    onLoaded: {
        // Получаем данные из C++
        var clipData = modelData

        // Устанавливаем позицию и размер
        item.x = clipData.startTime * root.pixelsPerSecond
        item.y = 5
        item.width = clipData.duration * root.pixelsPerSecond
        item.height = Theme.trackHeight - 10

        // Устанавливаем данные
        item.clipName = clipData.filename // ← Имя файла
        item.clipId = clipData.id
        item.selected = clipData.selected || false
        item.thumbnailPath = clipData.thumbnailPath || ""

        console.log("📁 VideoClip создан из C++:", clipData.filename,
            "x:", item.x.toFixed(1), "width:", item.width.toFixed(1))

        // TODO: Подключить сигналы перемещения
    }
}
```

ТЕСТИРОВАНИЕ:

1. Скомпилируй:

bash

Build → Clean All

Build → Rebuild

2. Запусти и drag видео:

В консоли должно быть:

🖥️ Timeline::addClip вызвана!

filepath: C:/.../video.mp4

✅ Длительность видео: 71.6568 секунд

✅ Клип добавлен! Индекс: 0 Всего клипов: 1

📡 Track 1 получил clipsChanged

🔄 Track 1 updateClipsFromCpp

📋 getClipsForTrack 1 → 1 клипов

Получено клипов: 1

🎬 VideoClip создан из C++: video.mp4 x: 0.0 width: 716.6 ← РЕАЛЬНАЯ ДЛИНА!

НЕ должно быть:

❌ qml: Mock клип добавлен ← Если видишь - mock НЕ убран!

🎨 АРХИТЕКТУРА ПОСЛЕ ИСПРАВЛЕНИЯ:

USER drag video.mp4

↓

Track.onClipDropped

↓

ClipManager.addClip()

↓

cppTimeline.addClip() ← C++!

↓

Timeline::addClip()

MediaDecoder.getDuration() → 71.6 сек

m_clips.append(newClip)

emit clipsChanged() ← Сигнал!

↓

Track.onClipsChanged() ← QML ловит!

↓

Track.updateClipsFromCpp()

↓

cppTimeline.getClipsForTrack(1) ← Запрос clips!

↓

Timeline::getClipsForTrack() ← C++ возвращает!

QVariantList with real data

↓

Track.realClips = [...] ← QML получает РЕАЛЬНЫЕ данные!

↓

Repeater creates VideoClip

filename: "video.mp4"

duration: 71.6 сек ← РЕАЛЬНАЯ!

width: 716 px



✓ ВИДЕО ОТОБРАЖАЕТСЯ ПРАВИЛЬНО!

✓ ЧЕКЛИСТ:

- ☐ Добавил `#include <QVariantList>` в `timeline.h`
- ☐ Добавил `Q_INVOKABLE QVariantList getClipsForTrack(int)` в `timeline.h`
- ☐ Добавил реализацию `getClipsForTrack()` в `timeline.cpp`
- ☐ Заменял `ClipManager.qml` (убрал `mock`)
- ☐ Исправил `Track.qml` (добавил `updateClipsFromCpp`)
- ☐ Скомпилировал
- ☐ Протестировал `drag` видео
- ☐ Убедился что `mock` НЕ вызывается

🎯 РЕЗУЛЬТАТ:

БЫЛО:

C++: ✓ Клип добавлен (реальная длина)
QML: ✗ Mock клип добавлен (10 сек)
UI: Неправильный размер клипа

СТАЛО:

C++: ✓ Клип добавлен (71.6 сек)
QML: ✓ Получил через `getClipsForTrack()`
UI: ✓ Правильный размер (716 px)!

СДЕЛАЙ ЭТИ ИЗМЕНЕНИЯ И КЛИПЫ БУДУТ ОТОБРАЖАТЬСЯ ПРАВИЛЬНО! 🎬 ✨